Object Oriented Programming using C++

www.infomax.org.in

Programming language

A **programming language** is a formal computer **language** designed to communicate instructions to a machine, particularly a computer. **Programming languages** can be used to create programs to control the behaviour of a machine or to express algorithms.

Machine oriented languages or low level language

A low-level language is a programming language that provides little or no abstraction of programming concepts, and is very close to writing actual machine instructions. Two good examples of low-level languages are assembly and machine code.

Problem Oriented language or High level language

A high-level language is a computer programming language that isn't limited by the computer, designed for a specific job, and is easier to understand. It is more like human language and less like machine language. However, for a computer to understand and run a program created with a high-level language, it must be compiled into machine language. Example: BASIC, C, C++, Cobol, FORTRAN, Java, Pascal, Perl, PHP, Python,

Compiler ,Interpreter and Assembler

- **Compiler :** A compiler is system software which converts programming language code into binary format in single steps. In other words Compiler is a system software which can take input from other any programming language and convert it into lower level machine dependent language.

- **Interpreter :** It is system software which is used to convert programming language code into binary format in step by step process.

- **Assembler:** An assembler is system software which is used to convert the assembly language instruction into binary format in step by step process. An assembler is system software which is used to convert the assembly language instruction into binary format.

Introduction to C++

- C++, as we all know is an extension to C language and was developed by ***Bjarne stroustrup*** at bell labs.
- C++ is an intermediate level language, as it comprises a confirmation of both high level and low level language features.
- C++ is a statically typed, free form, multiparadigm, compiled general-purpose language.
- C++ is an **Object Oriented Programming language** but is not purely Object Oriented.

C++ and Object Oriented Programming

Object Oriented programming is a programming style that is associated with the concept of Class, Objects and various other concepts revolving around these two, like Inheritance, Polymorphism, Abstraction, Encapsulation etc.

- **Class:** It is similar to structures in C language. Class can also be defined as user defined data type but it also contains functions in it. So, class is basically a blueprint for object. It declare & defines what data variables the object will have and what operations can be performed on the class's object.
- **Objects :** Objects are the basic unit of OOP. They are instances of class, which have data members and uses various member functions to perform tasks.

OOPS Concept

- **Abstraction:** Abstraction refers to showing only the essential features of the application and hiding the details. In C++, classes provide methods to the outside world to access & use the data variables, but the variables are hidden from direct access. This can be done access specifiers.
- **Encapsulation :** It can also be said data binding. Encapsulation is all about binding the data variables and functions together in class.
- **Inheritance :** Inheritance is a way to reuse once written code again and again. The class which is inherited is called base calls & the class which inherits is called derived class. So when, a derived class inherits a base class, the derived class can use all the functions which are defined in base class, hence making code reusable.
- **Polymorphism :** It is a feature, which lets us create functions with same name but different arguments, which will perform differently. That is function with same name, functioning in different way. Or, it also allows us to redefine a function to provide its new definition. You will learn how to do this in details soon in coming lessons.

Structure of C++ program

Include header file section
Global declaration section
main()
{
  Declaration part
  Executable part
}
User-defined functions
{
  Statements
}

C++ Character Set

Letters	Digits	White Space
Capital A to Z	All decimal digits 0 to 9	Blank Space
Small a to z		Horizontal tab
		Vertical tab
		New Line

Special Character

,	Comma	&	Ampersand
.	Dot	^	Caret
;	Semicolon	*	Asterisk
:	Colon	-	Minus
‘	Apostrophe	+	Plus
“	Quotation Mark	<	Less than
!	Exclamation mark	>	Greater than
|	Vertical bar	( )	Parenthesis
/	Slash	[ ]	Bracket
~	Tilde	%	Percent
_	Underscore	#	Hash
$	Dollar	=	Equal to
?	Question mark	@	At the rate

C++ tokens

C ++ tokens are the basic buildings blocks in C++ language which are constructed together to write a C++ program. Each and every smallest individual units in a C++ program are known as C++ tokens.

- Delimiter
- Keywords
- Identifiers
- Constants
- Strings
- Special Symbols
- Operators

Delimiters

A **delimiter** is a sequence of one or more characters used to specify the boundary between separate, independent regions in plain text or other data streams

Delimiters	Use
: Colon	Useful for label
; Semicolon	Terminate the statement
( ) parenthesis	Used in expression and function
[ ] Square bracket	Use for array declaration
{ } Curly Brace	Scope of the statement
# Hash	Pre-processor directive
, Comma	Variable separator

Keywords

Keywords are pre-defined words in a C++ compiler. Each keyword is meant to perform a specific function in a C++ program.

asm	else	operator	template	double
auto	enum	private	this	new
break	extern	protected	throw	switch
case	float	public	try	delete
catch	for	register	typedef	long
char	friend	return	union	struct
class	goto	short	unsigned	while
const	if	signed	virtual	
continue	inline	sizeof	void	
default	int	static	volatile	

Identifiers

- Identifiers are names of variables, functions, and arrays. They are user-defined names,
- consisting sequence of letters and digits, with the letter as the first character.

Rules for constructing identifier name in c:

- They must begin with a letter or underscore(_).
- They must consist of only letters, digits, or underscore. No other special character is allowed.
- It should not be a keyword.
- It must not contain white space.

Constant

Values do not change during the execution of the program

Numerical constants:

- **Integer constants:** These are the sequence of numbers from 0 to 9 without decimal points or fractional part or any other symbols.

  Eg: 10,20, + 30, – 14

- **Real constants:** It is also known as floating point constants.

  Eg: 2.5, 5.342

Character constants:

- **Single character constants**: A character constant is a single character. Characters are also represented with a single digit or a single special symbol or white space enclosed within a pair of single quote marks.

  Eg: ‘a’, ‘8’, ‘ ’.

- **String constants**: String constants are sequence of characters enclosed within double quote marks.

  Eg: “Hello”, “india”, “444”

Variable

It is a data name used for storing a data value. Its value may be changed during the program execution. The value of variables keeps on changing during the execution of a program.

Declaration of Variable:

<data type> <variable name>

Example;

int num;

Initialization on variable:

<data type> <variable name>= <value>;

Example:

int num=34;

char gen='m';

float amt=444.5;

Data Type

Data type	Size (Bytes)	Range	Format Specifiers
Char	1	– 128 to 127	%c
Unsigned char	1	0 to 255	%c
Short or int	2	– 32,768 to 32, 767	%i or %d
Unsigned int	2	0 to 655355	%u
Float	4	3.4e – 38 to +3.4e +38	%f or %g
Long	4	2147483648 to 2147483647	%ld
Unsigned long	4	0 to 4294967295	%lu
Double	8	1.7e – 308 to 1.7e+308	%lf
Long double	10	3.4e – 4932 to 1.1e+4932	%lf
Boolean			

Operators

An operator is a symbol that tells the compiler to perform specific mathematical or logical functions. C++ language is rich in built-in operators and provides the following types of operators :

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Increment and Decrement Operators
- Conditional Operators
- Special Operators

Arithmetic Operators

An arithmetic operator performs mathematical operations such as addition, subtraction and multiplication on numerical values (constants and variables).

Operator	Description	Example (int a=10,b=7;)
+	Addition	a+b =17
–	Subtraction	a-b = 3
*	Multiplication	a*b = 70
/	Division	a/b = 1
%	Modulus (Remainder)	a%b = 3
Note: modulus operator only used with int data type		

Relational Operators

A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.

Relational operators are used in decision making and loops.

Operator	Description	Example (int a=5,b=10)
>	greater than	a > b = 0
<	less than	a < b =1
>=	greater than or equal to	a >= b =0
<=	less than or equal to	a <= b =1
==	equal to	a == b =0
!=	not equal to	a != b =1

Logical Operators

Logical operators are used to combine two or more condition for building complex expression.

Operator	Description	Example
&&	Logical AND. True only if all operands are true	If a = 1 and b = 0 then (a && b) is False
||	Logical OR. True only if either one operand is true	If a = 1 and b = 0 then (a || b) is True
!	Logical NOT. True only if the operand is 0	If a = 1 then !(a) is False

Bitwise Operators

Bitwise operators perform manipulations of data at bit level. These operators also perform shifting of bits from right to left. Bitwise operators are not applied to float or double.

Operator	Description	Example: a=0,b=1
&	Bitwise AND	a&b =0
|	Bitwise OR	a|b =1
^	Bitwise exclusive OR	a^b =1
<<	Left Shift	
>>	Right Shift	

EXAMPLE: Bitwise Operators in C

a = 0001000
b= 2
a << b = 0100000
a >> b = 0000010

Assignment Operators

- An assignment operator is used for assigning a value to a variable. The most common assignment operator is =
- It assign the value from right to left side variable.

Operator	Description	Example
=	Simple Equal	C = A + B will assign the value of A + B to C
+=	Add & assignment	C += A is equivalent to C = C + A
- =	Subtract & assignment	C -= A is equivalent to C = C - A
*=	Multiply & assignment	C *= A is equivalent to C = C * A
/=	Divide & assignment	C /= A is equivalent to C = C / A
%=	Modulus & assignment	C %= A is equivalent to C = C % A

Increment/Decrement Operator

- Increment operators are used to increase the value of the variable by one and decrement operators are used to decrease the value of the variable by one in C programs.

Type of Increment Operator

- Pre increment/decrement Operator (++a, --a)
- Post increment/decrement Operator (a++, a--)

- In pre increment / decrement first increment/decrement the value of variable and then used inside the expression (initialize into another variable)
- In post-increment / decrement first value of variable is used in the expression (initialize into another variable) and then increment / decrement the value of variable.

Conditional Operator

The conditional operator (? :) is a ternary operator (it takes three operands).

Syntax:

expression ? expression : expression

The conditional operator works as follows:

- Expression1 is Condition
- Expression2 is Statement Followed if Condition is True
- Expression2 is Statement Followed if Condition is False

Precedence and Associativity of operators

- **Precedence:** If more than one operators are involved in an expression, C language has a predefined rule of priority for the operators. This rule of priority of operators is called operator precedence.

- **Associativity :** If two operators of same precedence (priority) is present in an expression, Associativity of operators indicate the order in which they execute.

Operator	Description	Associativity
( ) [ ] . ->Member selection via pointer ++ --	Parentheses (function call) (see Note 1) Brackets (array subscript) Member selection via object name Postfix increment/decrement (see Note 2)	left-to-right
++ -- + - ! ~ (*type*) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of *type*) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
|	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
||	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

Comments in C++

Generally **Comments** are used to provide the description about the Logic written in program. Comments are not display on output screen.

When we are used the comments, then that specific part will be ignored by compiler.

In 'C++' language two types of comments are possible

- Single line comments
- Multiple line comments

Single line comments

Single line comments can be provided by using / /.....

Multiple line comments

Multiple line comments can be provided by using /*.....*/

The standard output stream (cout<<)

The predefined object **cout** is an instance of **ostream** class. The cout object is said to be "connected to" the standard output device, which usually is the display screen. The **cout** is used in conjunction with the stream insertion operator, which is written as << which are two less than signs as shown in the following example.

```
#include <iostream>
void main( )
{
    int age=23;
    cout << “Your Age is: " << age<< endl;
}
```

The standard input stream (cin>>)

The predefined object **cin** is an instance of **istream** class. The cin object is said to be attached to the standard input device, which usually is the keyboard. The **cin** is used in conjunction with the stream extraction operator, which is written as >> which are two greater than signs as shown in the following example.

```
#include <iostream.h>
void main( )
 {
        int age;
         cin >> age;
}
```

Type Casting

Type casting is process to convert a variable from one data type to another data type. For example if we want to store a integer value in a float variable then we need to typecast integer into float.

There are tow type of conversion

- Implicit
- Explicit

Implicit: When the type conversion is performed automatically by the compiler without programmers intervention, such type of conversion is known as **implicit type conversion**

Explicit: The type conversion performed by the programmer by posing the data type of the expression of specific type is known as explicit type conversion. The explicit type conversion is also known as **type casting**.

Syntax:

```
(data_type) var_name;
```

Example:

```
int =65;
char ch=(char)a;
```

Flow Control Statement

The statements inside your source files are generally executed from top to bottom, in the order that they appear. *Control flow statements*, however, break up the flow of execution by employing decision making, looping, and branching, enabling your program to *conditionally* execute particular blocks of code.

- Conditional Statement
- Looping statement
- Jumping Statement

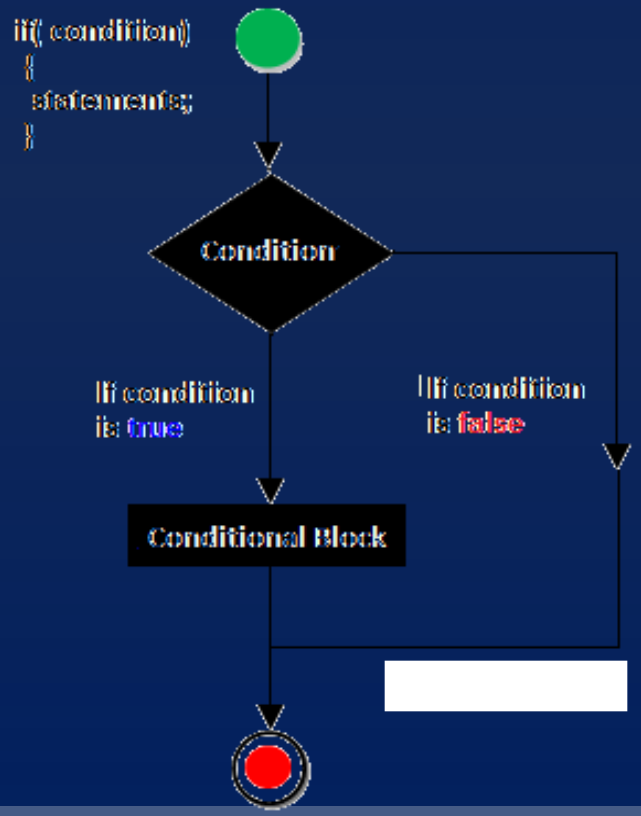
Conditional statement

- **Decision making statement** is depending on the condition block need to be executed or not which is decided by condition. In C programming language there are three types of conditional statement:

- If statement
- If else statement
  - Nested if else statement
  - Else if ladder statement
- Switch case statement

if Statement

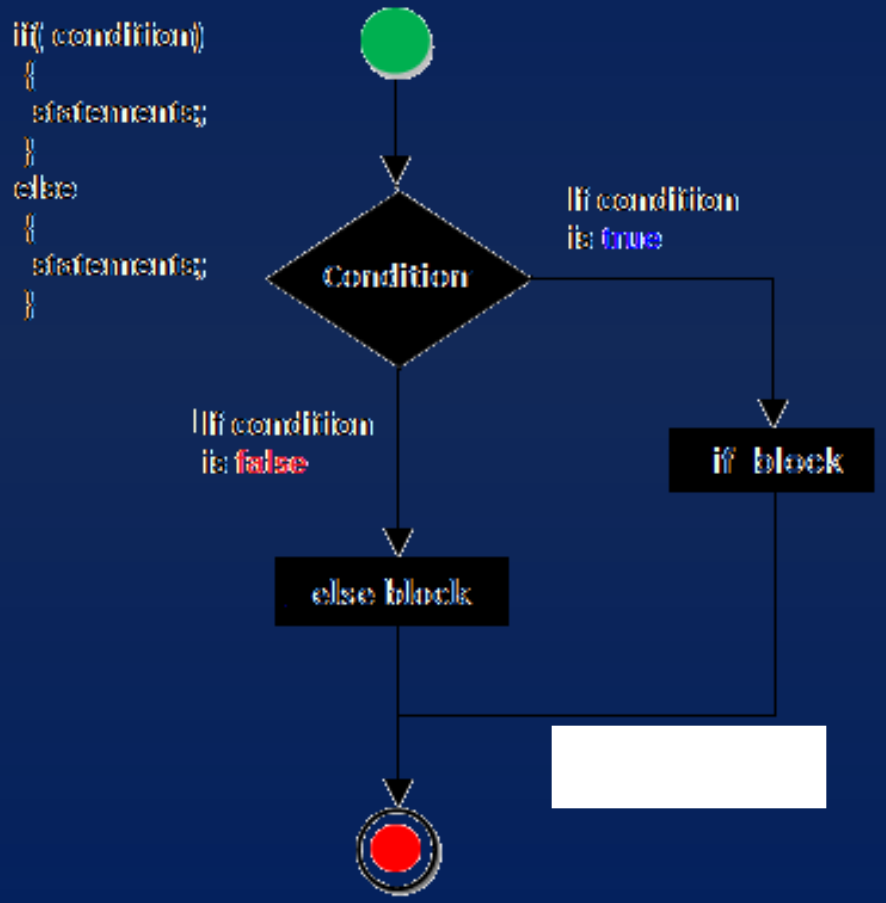
- If statement is most basic statement of Decision making statement. It tells to program to execute a certain part of code only if particular condition is true.




```
void main()
{
        int num;
        cout<<“enter any number”;
        cin>>num;
        if(num<0)
        {
                num=-num;
        }
        cout<<“absolute value:”<<num;
        getch()
}
```

if-else statement

In general it can be used to execute one block of statement among two blocks, in C language if and else are the keyword in C.



Example:

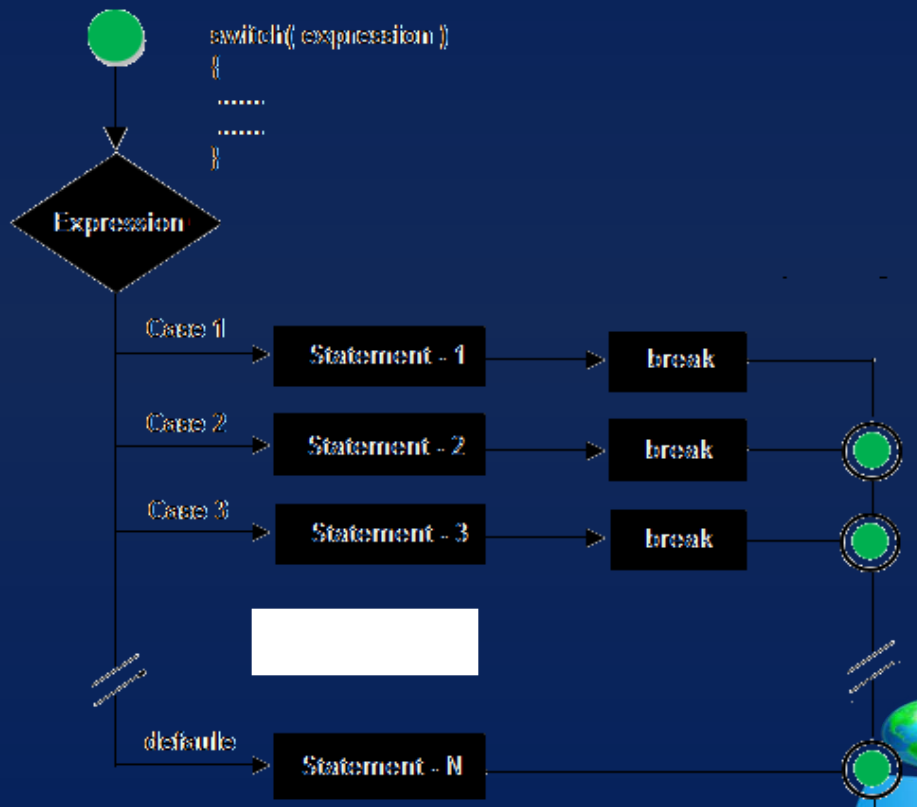
```
void main()
{
        int num;
        cout<<“Enter  any int Number”;
        cin>>num;
        if(num%2==0)
        {
                cout<<Ëven Number”;
        }
        esle
        {
                cout<<“”Number is ODD”;
        }
        getch();
}
```

Switch Statement

A switch statement work with byte, short, char and int primitive data type, it also works with enumerated types and string.

Syntax:

```
switch(expression)
{
        case  constant:
                statement;
        break;
        case  constant:
                statement;
        break;
        ..
        ..
        default:
                statement;
}
```



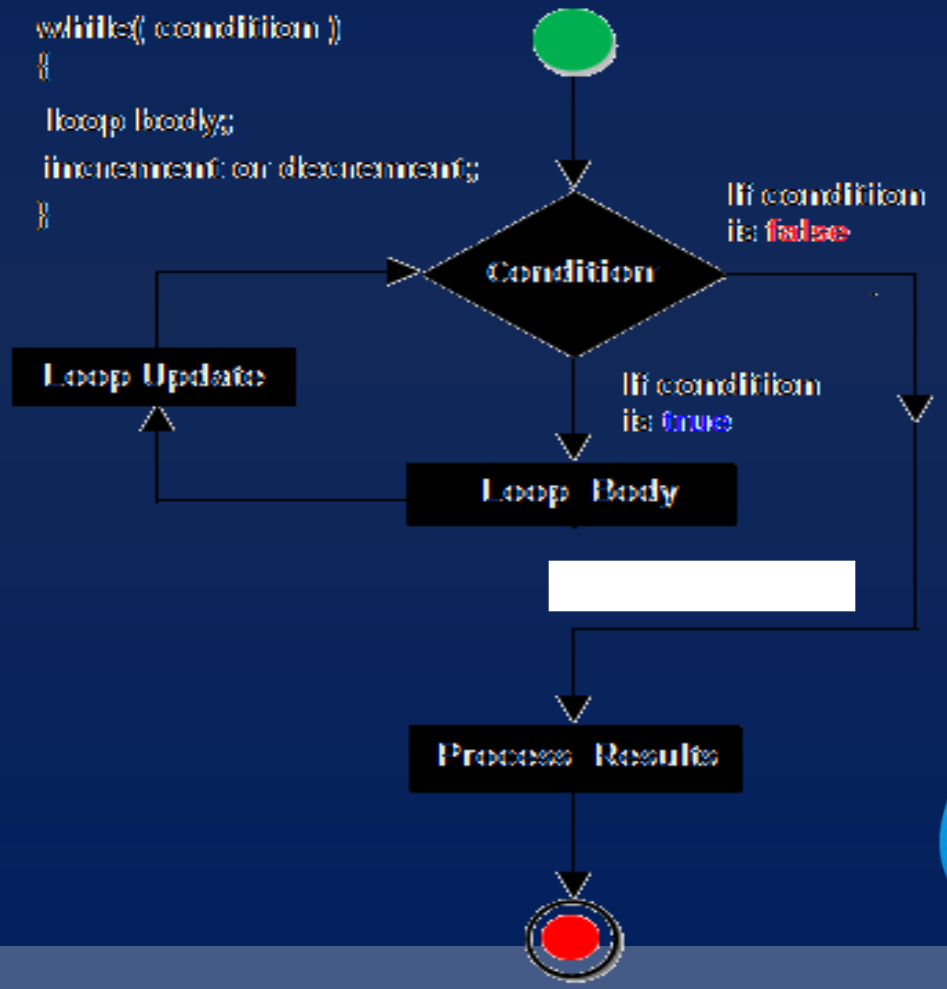
Looping statement

- **Looping statement** are the statements execute one or more statement repeatedly several number of times. In C programming language there are three types of loops:

- For loop
- While loop
- Do while loop

While loop

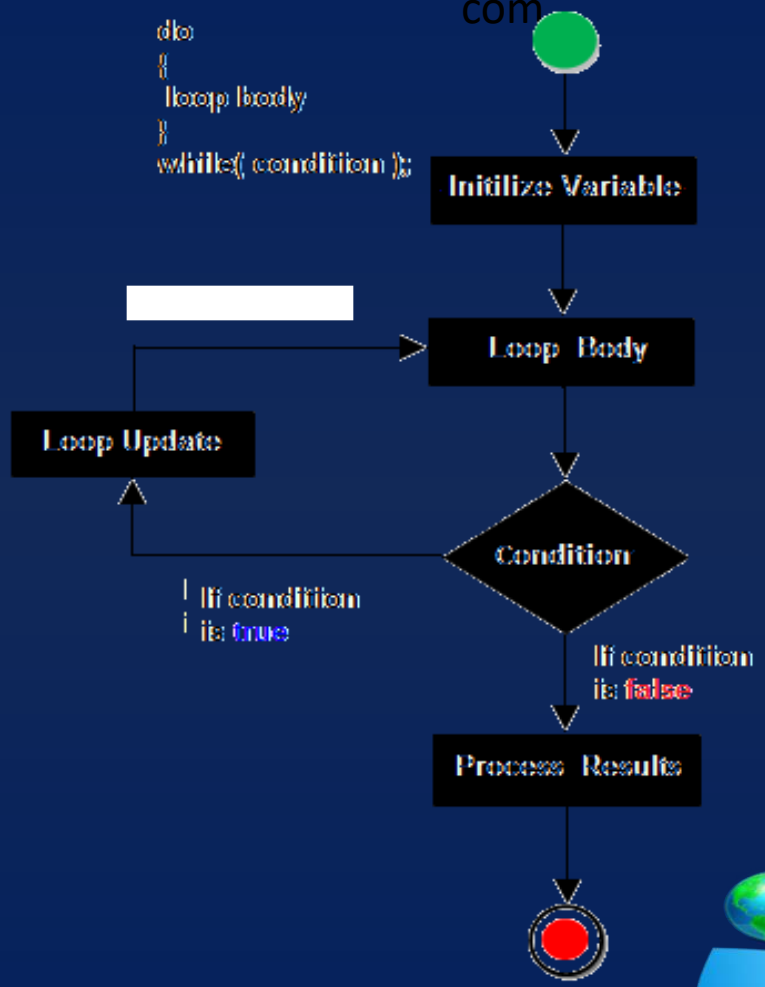
In **while loop** First check the condition if condition is true then control goes inside the loop body other wise goes outside the body. **while loop** will be repeats in clock wise direction.



do-while

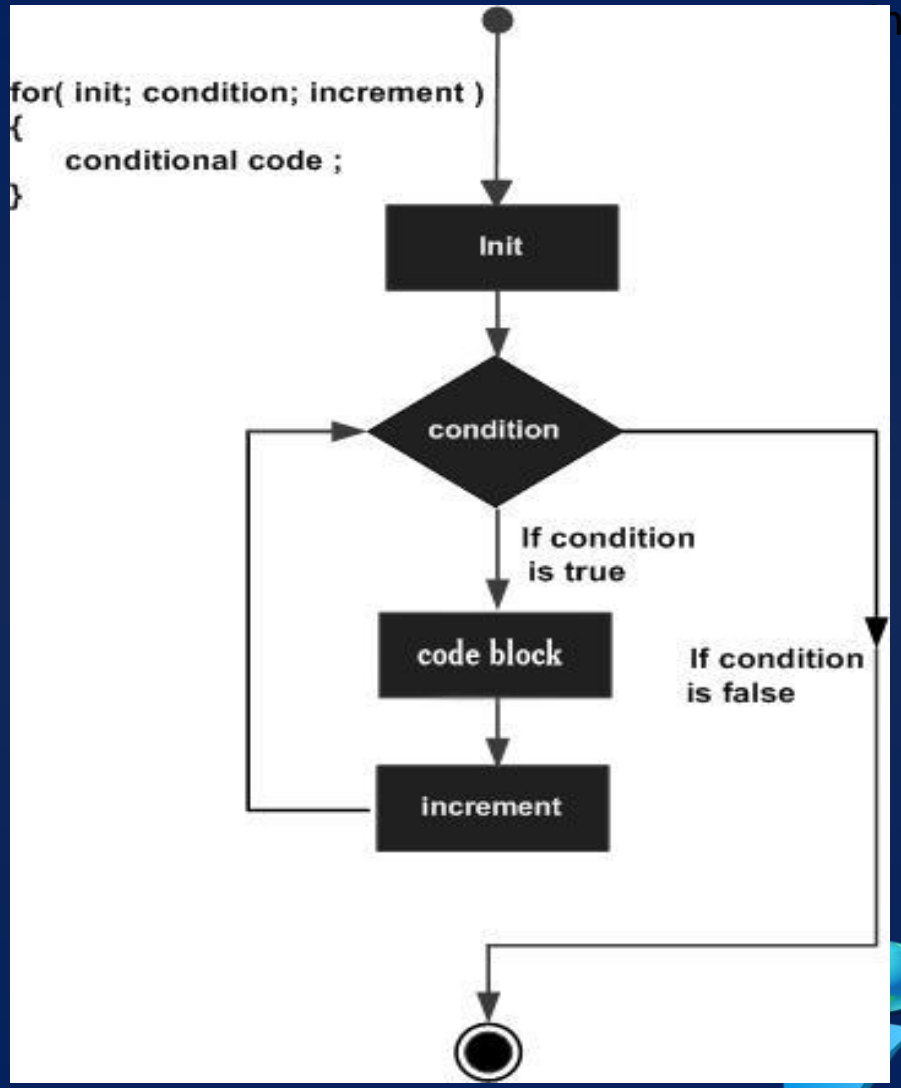
A **do-while** loop is similar to a while loop, except that a do-while loop is execute at least one time.

A do while loop is a control flow statement that executes a block of code at least once, and then repeatedly executes the block, or not, depending on a given condition at the end of the block (in while).



For loop

for loop is a statement which allows code to be repeatedly executed. For loop contains 3 parts Initialization, Condition and Increment or Decrements.



Jumping Statement

C language provides us multiple statements through which we can transfer the control anywhere in the program.

There are basically 3 Jumping statements:

1. break jumping statements.
2. continue jumping statements.
3. goto jumping statements.
4. return Statement

Break Statements

- By using this jumping statement, we can terminate the further execution of the program and transfer the control to the end of any immediate loop or switch body.
- To do all this we have to specify a break jumping statements whenever we want to terminate from the loop.

- **Syntax:** break;

- **NOTE:** This jumping statements always used with the control structure like switch case, while, do while, for loop etc.

  NOTE: As break jumping statements ends/terminate loop of one level . so it is refered to use return or goto jumping statements , during more deeply nested loops.

Continue Statements.

- By using this jumping statement, we can terminate the further execution of the program and transfer the control to the begining of any immediate loop.
- To do all this we have to specify a continue jumping statements whenever we want to terminate terminate any particular condition and restart/continue our execution.

- **Syntax:** continue;

- **NOTE:** This jumping statements always used with the control structure like switch case, while, do while, for loop etc.

Goto Statements

- By using this jumping statements we can transfer the control from current location to anywhere in the program.
- To do all this we have to specify a label with goto and the controlwill transfer to the location where the label is specified.

Syntax:

```
<label>:
        statement
goto <label>;
```

Note :

- It is good programming style to use the break, continue and return instead of goto.
- However, the break may execute from single loop and goto executes from more deeper loops.

Return Statement

- return statement is used to transfer program's control from called function to calling function, it's secondary task is to carry value from called function to calling function.
- If function's return type is void, you can use return only to return program's control to the calling function, and if there is a return type you will have to return same type of value to the calling function.

Syntax:

```
return;
```

Or

```
return (value)
```

Array

An array is a collection of variable of the same data type that is referred a common name. Each element in array is associated with an index number and occupies contiguous memory location.

Characteristics of an array:

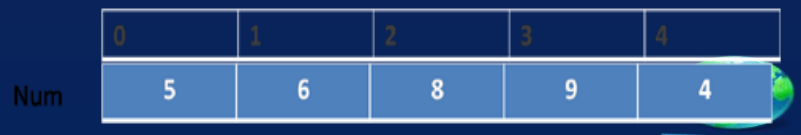
- It is a collection of similar (homogeneous) data type of element.
- The array index number always starts with 0(zero) so the last index is size-1.
- It always stored in contiguous memory location.
- The size and dimension of array is fixed.
- It is a linear data structure that stores the element in a sequence.

Declaration of an array:

<Data_type> <array_name>[size];

Example:

int num[5];



Initialization of an array:

int num[5]={4,5,6,5,4,3};

String in C

Strings are actually one-dimensional array of characters terminated by a **null** character '\0'.

Declaration :

char <array_name>[size];

Example:

char name[20];

Initialization :

char greeting[] = "Hello";

OR,

char greeting[] = {‘H', ‘e', ‘l', ‘l‘, ’o’, '\0'};

Input output String:

gets(): Input String

puts(): Output/Print String

String Function

Functions	Description
strcat ( )	strcat (str2, str1); – str1 is concatenated at the end of str2.
strcpy ( )	strcpy (str1, str2) – It copies contents of str2 into str1.
strlen ( )	int x=strlen(str1) it counts the number of characters in str1
strcmp ( )	int x=strcmp(str1,str2) : Returns 0 if str1 is same as str2. Returns <0 if strl < str2. Returns >0 if str1 > str2
strlwr ( )	strlwr(str1) Converts string to lowercase
strupr ( )	strupr(str1) Converts string to uppercase
strrev ( )	strrev(str1) Reverses the given string

Function in C++

A function is a group of statements that together perform a task.

Types of functions in C programming

- Standard library functions
- User defined functions

Defining a Function

```
return_type function_name( parameter list )
{
    body of the function
}
```

Return Type: The **return_type** is the data type of the value the function returns , if function does not return value the it will be void.

Function Name – This is the actual name of the function.

Parameters : The parameter list refers to the type, order, and number of the parameters of a function.

Function Body – The function body contains a collection of statements that define what the function does.

Structure

Structure is a collection of variables of different types under a single name.

Structure Definition in C

Keyword struct is used for creating a structure.

Syntax of structure

```
struct structure_name
 {
    data_type member1;
   data_type member2;
   . .
    data_type memeber;
 };
```

Structure variable declaration:

struct <stucture_name> <var_name>;

Example:

```
struct person
{
 char name[50];
 int citNo;
 float salary;
 };

Void main()
{
  struct person person1;
  ……
}
```